

EE5203 Lab 3 Guide

Functions and registers: a sensor module and an LED driver for LD2 - Basri Erdogan

Mission: Move your Lab 2 calculations into C functions, and write a small LED driver that controls LD2 (pin PA5) through CMSIS registers. At the end, LD2 blinks the status of your eight sensor values: **1 blink = LOW, 2 blinks = OK, 3 blinks = HIGH**.

This lab uses Lecture 3 (functions, header files, `CMakeLists.txt`) and Lecture 4 (registers, bit masks, `->`).

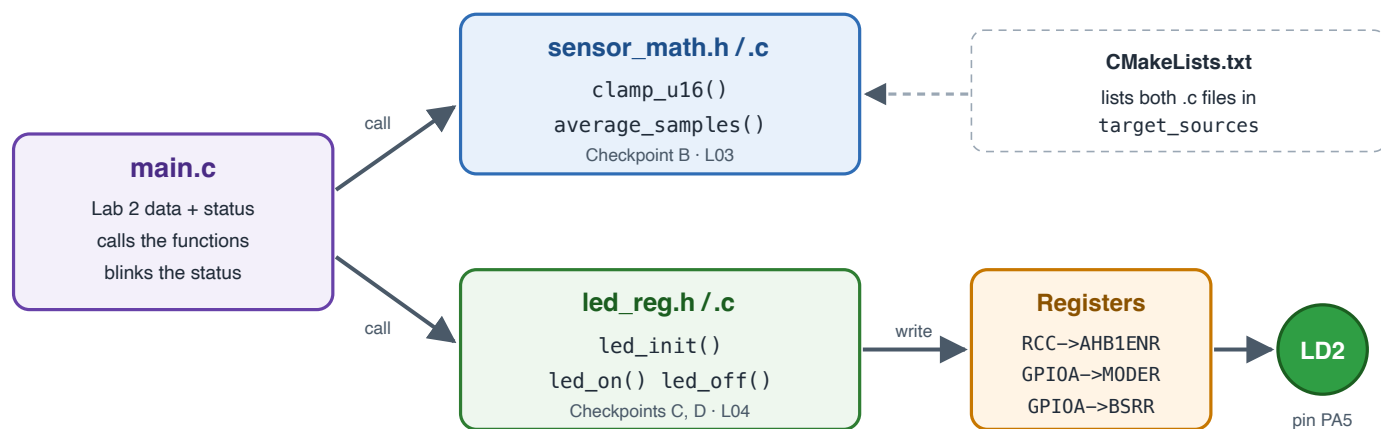


Figure 1: What you build: `main.c` calls two small modules; the LED module writes three registers that drive LD2.

Prepare before class

- ☐ Bring your board and a USB data cable. Your Lab 2 project must be on `H:\EE_Labs`: you copy code from it, but you do not change it.
- ☐ Download the starter files: [EE5203_L03_starter.zip](#).
- ☐ Do the **Lab 3 pre-lab quiz** on CATS. It includes the bit-mask table below. To practise masks first, use the [Bitwise Lab](#).
- ☐ Bring this table, completed on paper:

Purpose	Expression	Hex value
GPIOA clock bit	<code>1U << 0</code>	
PA5 mode field (two bits)	<code>3U << 10</code>	
PA5 output pattern 01	<code>1U << 10</code>	
Set PA5	<code>1U << 5</code>	
Reset PA5	<code>1U << 21</code>	

Learning objectives

By checkoff time, you can:

- write a function declaration in a `.h` file and its definition in a `.c` file;
- add a new `.c` file to the build in `CMakeLists.txt`;
- replace repeated code in `main.c` with function calls and get the same results;
- set and clear register bits with `|=`, `&=` and masks;
- show in the debugger that a register value agrees with what LD2 does.

Hardware and files

Item	Value
Board	Nucleo-F401RE
Tools	STM32CubeMX (new project) + VS Code
Starting point	A new CubeMX project; code copied from your Lab 2 <code>main.c</code>
Project name	<code>EE5203_L03_<studentnumber></code>
New files (starter)	<code>sensor_math.h</code> , <code>sensor_math.c</code> , <code>led_reg.h</code> , <code>led_reg.c</code>
Files you edit	<code>CMakeLists.txt</code> , <code>main.c</code> , and the four new files
Registers	<code>RCC->AHB1ENR</code> , <code>GPIOA->MODER</code> , <code>GPIOA->BSRR</code>

Keep all edits to `main.c` inside the `USER CODE` regions.

Checkpoint A - Create a new project and add the starter files

Start of the session: if `C:\Work\<studentnumber>` does not contain your Lab 2 project, copy **all** your project folders from `H:\EE_Labs` into it first (as in Lab 2). Do not run *Save to H* now.

You make a **new** project. Do not change your Lab 2 project.

1. In **STM32CubeMX**: Board Selector → **NUCLEO-F401RE** → accept the default initialization (as in Lab 1). PA5 is already LD2.
2. Project Manager: name **EE5203_L03_<studentnumber>**, location `C:\Work\<studentnumber>`, **Toolchain/IDE = CMake** → Generate Code.
3. Copy the folder `C:\Users\Public\EE5203_template\.vscode` into the new project folder, next to the `.ioc` file.
4. Open `EE5203_L03_starter.zip`. Copy the **Core** folder inside it and paste it into your project folder. Two files go to `Core\Inc`, two to `Core\Src`.
5. In `CMakeLists.txt`, add the two new `.c` files to `target_sources`, below the line `# Add user sources here`. Write the paths yourself, in the same form as `Core/Src/<file>.c`. (The project name is already correct: CubeMX used your folder name.)
6. Open the folder in VS Code and build with **Ctrl+Shift+B**: zero errors. The starter files build even though they are not finished.

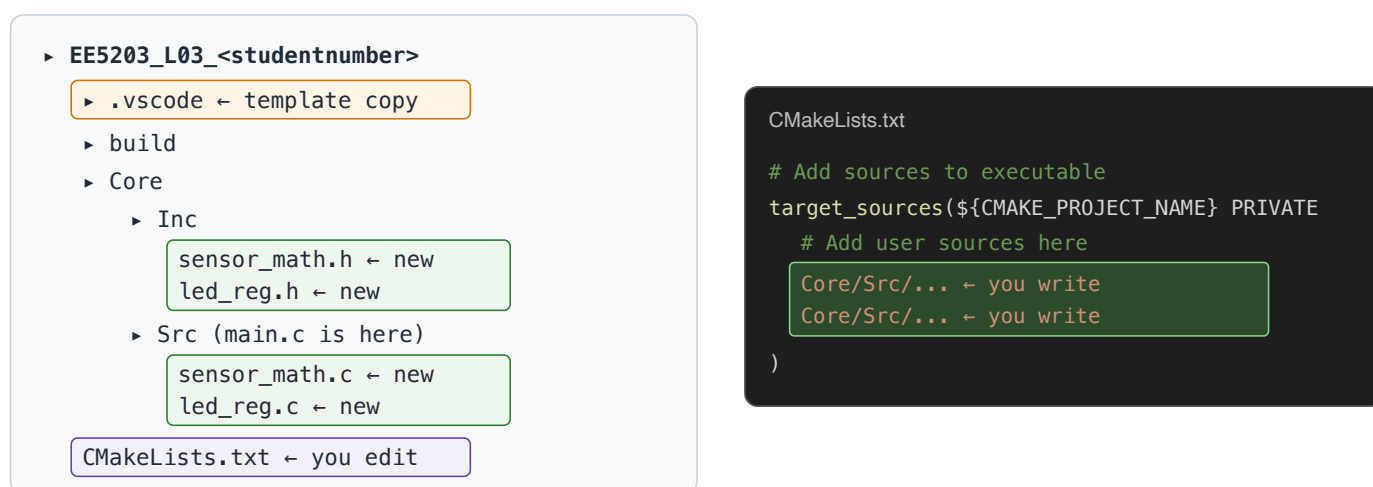


Figure 2: Your project folder after Checkpoint A.

Show: the four new files in the Explorer, your two lines in `CMakeLists.txt`, and the build line **Linking C executable** `EE5203_L03_<studentnumber>.elf`.

Checkpoint B - Move the Lab 2 calculations into functions

`sensor_math.h` already declares `clamp_u16`. You write the rest.

1. In `sensor_math.c`, complete the **body** of `clamp_u16`: return low if the value is below low, high if it is above high, and the value itself otherwise.
2. In `sensor_math.h`, **write the declaration** of `average_samples`. It takes an array of `uint16_t` values and the number of values (`int`), and returns the average as `uint16_t`.
3. In `sensor_math.c`, **write the definition** of `average_samples`:
 - return 0U when the count is 0;
 - add the values into a `uint32_t` sum; the loop uses `i < count`;
 - divide once, at the end.
4. In `main.c`, add `#include "sensor_math.h"` inside `USER CODE BEGIN Includes`.
5. Open your **Lab 2** `main.c` (read only). Copy everything between `USER CODE BEGIN 2` and `USER CODE END 2` into the same region of the new `main.c`. Then change the copy:
 - replace the clamping if with a call: `samples[i] = clamp_u16(samples[i], 0U, 4095U);`
 - remove the sum line from the loop, and after the loop write `average = average_samples(samples, 8);`
 - keep the `high_count` and `status_code` code from Lab 2.

The results must be **the same as in Lab 2**. Fill in the table from your Lab 2 prediction table, then check in the debugger:

Quantity	Lab 2 value	Lab 3 value (debugger)
<code>samples[3]</code> after the loop		
<code>average</code>		
<code>high_count</code>		

Quantity	Lab 2 value	Lab 3 value (debugger)
status_code		

Show: a breakpoint inside the loop of `average_samples`, with `i` and the sum changing as you step, and the completed table.

Checkpoint C - Write `led_init()` and check the registers

`led_reg.h` and `led_reg.c` contain only comments. You write the code. The three registers you use in Checkpoints C and D:

RCC->AHB1ENR

bit 0 = GPIOA clock on · use `1` = (keep the other bits)



GPIOA->MODER

bits 11:10 = PA5 mode · 00 input, 01 output



PA5 starts at bit $5 \times 2 = 10$: clear with `&= ~(3U << 10)`, then set with `1 = (1U << 10)`

GPIOA->BSRR

write-only · use `=` (a 1 acts, a 0 does nothing)



bits 31–16: RESET pins (bit 21 = $5 + 16 \rightarrow$ PA5 off)

bits 15–0: SET pins (bit 5 $\rightarrow$ PA5 on)

Figure 3: The bits you change: AHB1ENR bit 0, MODER bits 11:10 for PA5, BSRR bit 5 (set) and bit 21 (reset).

1. In `led_reg.h`, **declare** three functions with no parameters and no return value: `led_init`, `led_on`, `led_off`.
2. In `led_reg.c`, **define** `led_init` with three statements:
 - set bit 0 of RCC->AHB1ENR (GPIOA clock) without changing other bits;
 - clear bits 11:10 of GPIOA->MODER;
 - write the pattern 01 into bits 11:10 (PA5 = output).
3. In `main.c`, add `#include "led_reg.h"`. At the end of `USER CODE BEGIN 2`, add these two lines:

```
GPIOA->MODER &= ~(3U << 10); /* start from input mode: makes the change visible */
led_init();
```

CubeMX already set up PA5 for you. The first line undoes that, so you can see that your `led_init` really does it.

Break on `led_init()`; , then step **into** it (F11) and over each statement. Look at GPIOA and RCC in the register (SFR) view:

Moment	RCC->AHB1ENR bit 0	MODER bits 11:10
Before <code>led_init</code>		
After the clear		
After the set		

Show: the table, and explain why bit 0 was already 1 before `led_init`.

Checkpoint D - Turn LD2 on and off with BSRR

1. In `led_reg.c`, **define** `led_on` and `led_off`. Each is one assignment to GPIOA->BSRR: bit 5 sets PA5, bit 21 ($5 + 16$) resets it.
2. In `main.c`, inside `USER CODE BEGIN 3`, write a blink with `led_on()`, `led_off()` and `HAL_Delay(200)`. Do not use any `HAL_GPIO_...` call.
3. Put a breakpoint on `led_off()`; . When it stops, LD2 must be **on**. Step over: LD2 turns **off**.

Show: LD2 blinking, your three functions, and the register view when the program stops at `led_off()`; (`ODR` bit 5 = 1).

Checkpoint E - Blink the status

Use `status_code` from Checkpoint B (-1, 0 or 1).



Figure 4: The three blink patterns.

1. Calculate a blink count: LOW = 1, OK = 2, HIGH = 3. One short expression is enough.
2. In `USER CODE BEGIN 3`, blink LD2 that many times (200 ms on, 200 ms off), then wait 1000 ms. Use a `for` loop.
3. Predict the number of blinks for your data before you run it.

Show: the blink pattern, your prediction, and the code.

If you finish early: change one of your samples so that the status becomes HIGH, predict the new blink count, and check it.

Common problems

What you see	First check
undefined reference to 'average_samples' or 'led_init' implicit declaration of function 'led_on'	The <code>.c</code> file is in <code>target_sources</code> . The name in the <code>.c</code> file matches the <code>.h</code> file. <code>#include "led_reg.h"</code> is in <code>main.c</code> , and the declaration is in the header.
'GPIOA' undeclared in <code>led_reg.c</code> conflicting types for 'average_samples'	<code>#include "main.h"</code> is at the top of <code>led_reg.c</code> . The declaration and the definition use exactly the same types.
LD2 never lights after <code>led_init</code> LD2 stays on The average differs from Lab 2 ...FLASH.ld: syntax error	Check the <code>MODER</code> bits in the register view: they must be 01. <code>led_off</code> uses bit 21, not bit 5. Stop inside <code>average_samples</code> ; watch <code>i</code> , <code>count</code> and the sum. Build with Ctrl+Shift+B (the template repairs the CubeMX linker script). Check that <code>.vscode</code> is in the project folder.

End of the session - Save to H

Close VS Code, run the **Save to H** desktop shortcut, and check that `H:\EE_Labs\EE5203_L03_<studentnumber>` exists before you log out.

Acceptance checklist

- ☐ The build is clean. Every declaration matches its definition.
- ☐ Lab 3 results equal the Lab 2 results (Checkpoint B table), and the Lab 2 project is unchanged.
- ☐ `led_init`, `led_on`, `led_off` use only register writes; there is no `HAL_GPIO_...` call in your code.
- ☐ The register table of Checkpoint C is filled in.
- ☐ LD2 blinks the status (Checkpoint E).
- ☐ All `main.c` edits are inside the `USER CODE` regions.

Individual oral check

Be ready for one question and **one live change**, chosen by your lab assistant. Predict the result first, then make the change.

Questions:

- What is the difference between a declaration and a definition? Where is each one for `led_on`?

- Why does `average_samples` need the count as a parameter?
- Why does `led_init` use `!=` for the clock, but `led_on` uses `=` for `BSRR`?
- What does `3U << 10` select, and why does PA5 start at bit 10?

Live changes:

- Change both blink delays to 100 ms. Do the masks change?
- Change the HIGH limit from 3000 to 2000. How many blinks now?
- Replace the literal 5 with `#define LED_PIN 5U` in `led_on` and `led_off`.

Submit

Submit the four module files, `CMakeLists.txt`, the `main.c` user-code sections, the Checkpoint B and C tables, and one photo or short video of LD2 blinking the status.

Marks (10): pre-lab quiz on CATS 1 · A new project and build 1 · B functions 2 · C `led_init` and registers 2 · D `led_on/led_off` 2 · E status blink 1 · oral check and live change 1.